

Robust, Scalable, Nash-Equilibrium-Seeking Machine Learning Algorithms for Adversarial Agents in a Decentralized Collaborative Network

Team 2026160: Aaron Zhou, Elwin Cheng, Jingmin Wang
Supervisor: Lacra Pavel Administrator: Afshin Poraria

Introduction

In the real world, systems are made up of many decision makers (agents), all trying to maximize benefit for themselves. The **Nash Equilibrium (NE)** is a system state where no agent has anything to gain by changing only their action and/or strategy. This is state is usually desirable as systems become more stable and predictable

However, real world systems are increasingly **decentralized**. This means that there is no central controller for all agents; agents may only communicate with a certain number of neighbors. Furthermore, certain agents may be **adversarial**, meaning that they emit random noise, lie, or other disturbances that disrupt NE-seeking algorithms. Therefore, it is paramount to develop robust NE-seeking algorithms that can succeed despite these conditions

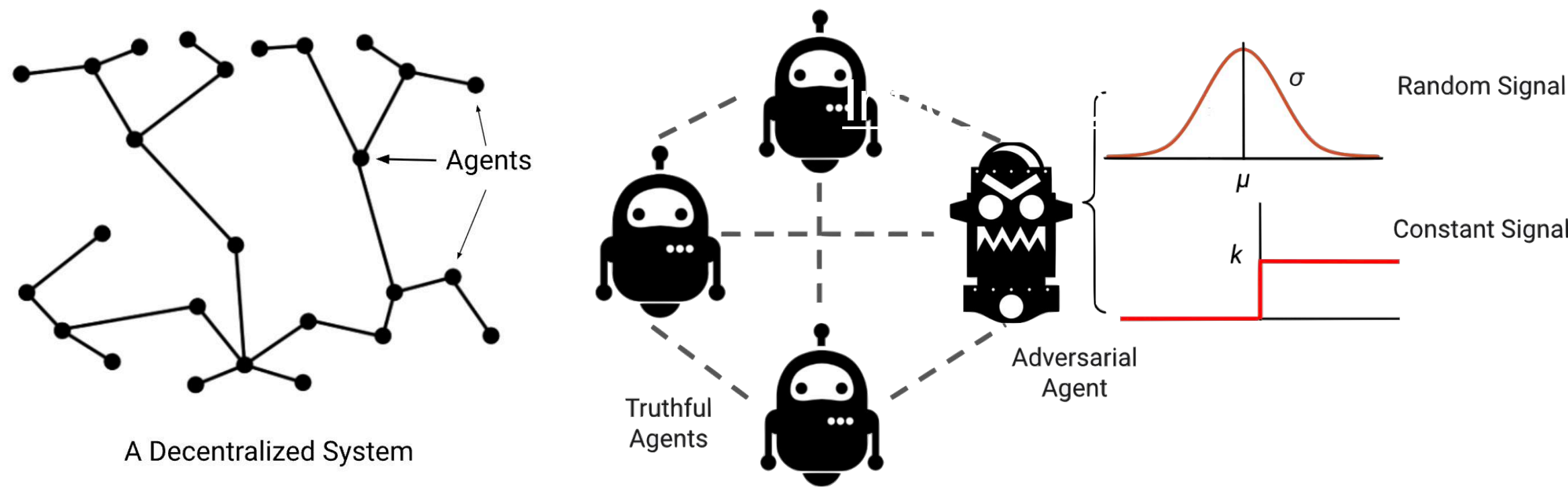


Figure 1. Examples of a decentralized system and adversarial agents. **Left:** An example of a decentralized system, where agents can only communicate with certain neighbors. **Right:** An example of an adversarial agent in a system. In our model, adversarial agents can “lie” via emitting random noise or a constant disturbance

Our Contributions

We improve on a **baseline method** developed by Gadjov and Pavel. We outline our major improvements over the original method here:

- 1) We developed an algorithm more robust to adversarial messages by using the **geometric median**
 - 2) We extended the original problem setup to include **time-varying graphs** (the graphs by which agents communicate change over time)
 - 3) We extended the original problem setup to support **constrained optimization** (agents are not allowed to take certain actions or communicate certain messages)
 - 4) We made the original algorithm more **scalable** in memory and runtime.
 - 5) We implemented **adaptive gradient play** algorithm framework for faster convergence
- We integrate these improvements into a **unified algorithm** in the System Block Diagram

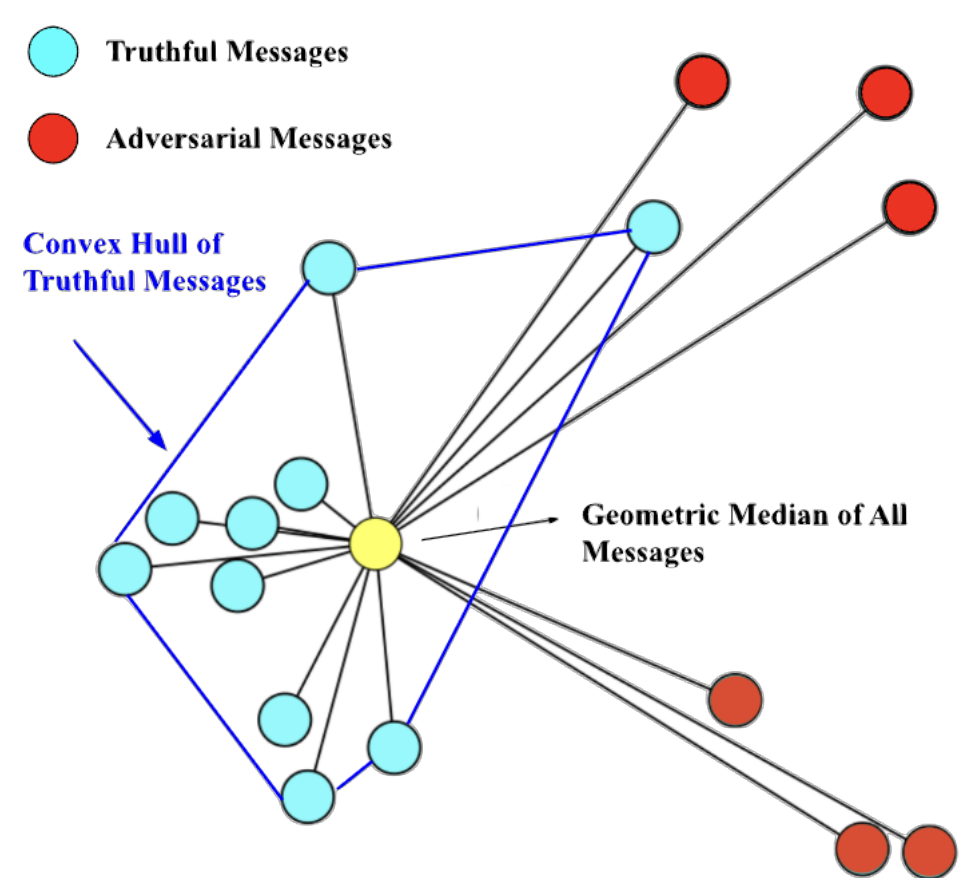


Figure 2. A representation of the geometric median. Even when there are extreme values (i.e. the red adversarial messages), the geometric median lies “close” to the light blue truthful messages so long as there are more truthful messages than adversarial ones. Usually, the geometric median lies in the convex hull of truthful messages. This forms the bases of our geometric median aggregator

System Block Diagram

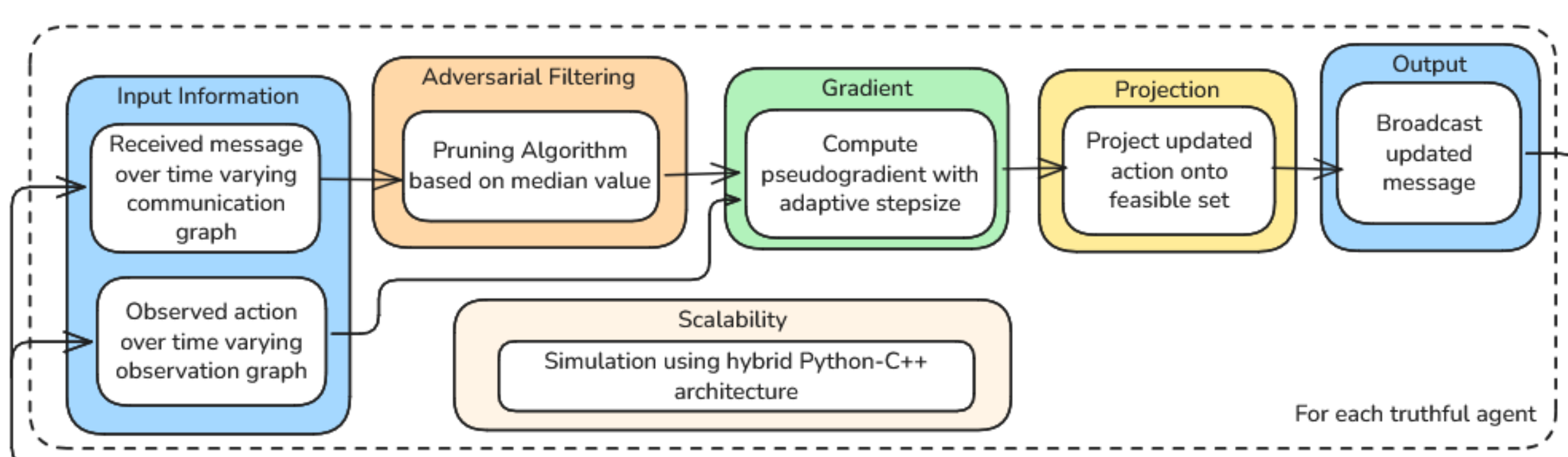


Figure 3. Block diagram depicting our algorithm (for every truthful agent at every timestep)

Scalability

Adversarial Filtering within each agent is independent of other agents, making it embarrassingly parallel. However, Python’s Global Interpreter Lock (GIL) prevents true multithreading and incurs significant interpreter overhead. We chose a hybrid Python–C++ approach because it provides the best balance of performance and practicality. Our tests show a 4x speedup in program execution over the original Python implementation, shown in Fig. 4

The original algorithm computed a matrix product $(R^T F)v$. The intermediate product $R^T F$ is a dense matrix with $O(N^4)$ entries, where N is the number of agents. This makes larger simulations computationally infeasible. We exploit the associativity of matrix multiplication and decompose the computation into $R^T(Fv)$. The largest matrix that must be held in memory is R^T itself, which has $O(N^3)$ entries. This enabled the simulation to scale from 165 agents to over 1,000, shown in Fig. 5.



Figure 4. The runtime of the improved algorithm is 4x faster even as more agents are simulated

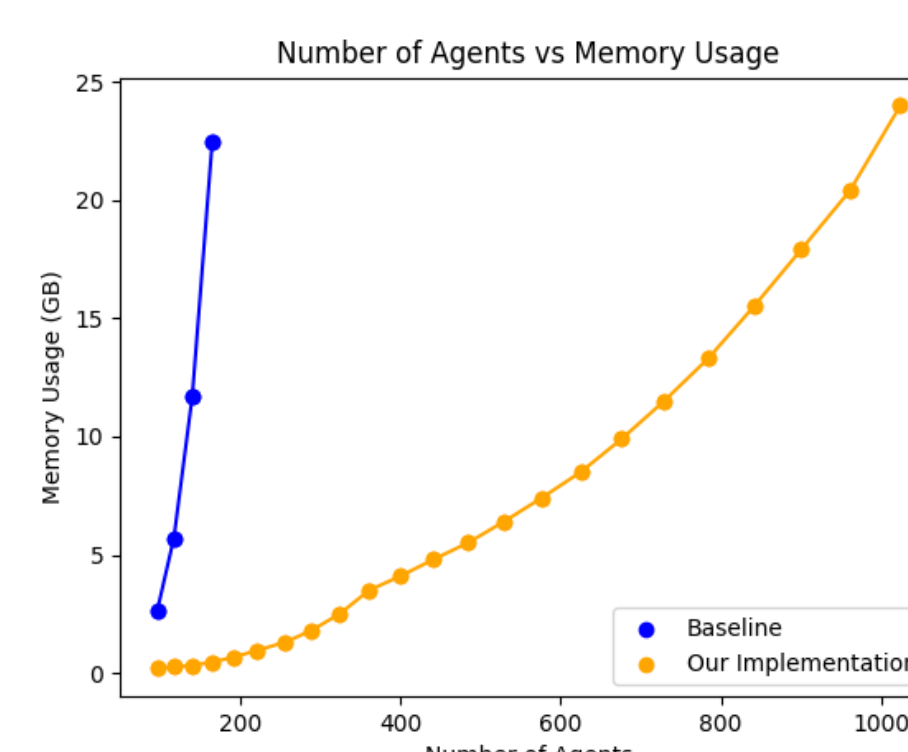


Figure 5. Memory usage of the improved algorithm grows an order of magnitude slower than the original

Geometric Median Aggregator

In the baseline algorithm, to do adversarial filtering, they rely on a human-tuned hyperparameter named “D” – when receiving messages, they remove the $2 \times D$ most extreme values. However, there are two major shortcomings of this method.

- 1) If the number of adversarial agents is greater than D , then the pruning does not remove all the adversarial agents, preventing convergence to the NE
- 2) If there are adversarial agents remaining, then the magnitude of the adversarial signal will affect the performance of the algorithm
- 3) Therefore, a human must know beforehand how many adversarial agents there are and the magnitude of their messages, which is impractical.

Our method, which takes the geometric median of the incoming messages, has none of the problems above. It does not rely on a hyper parameter, and so there is no risk of neglecting any adversarial agents or necessitating a human having a priori knowledge. It is also robust to the magnitude of the adversarial messages.

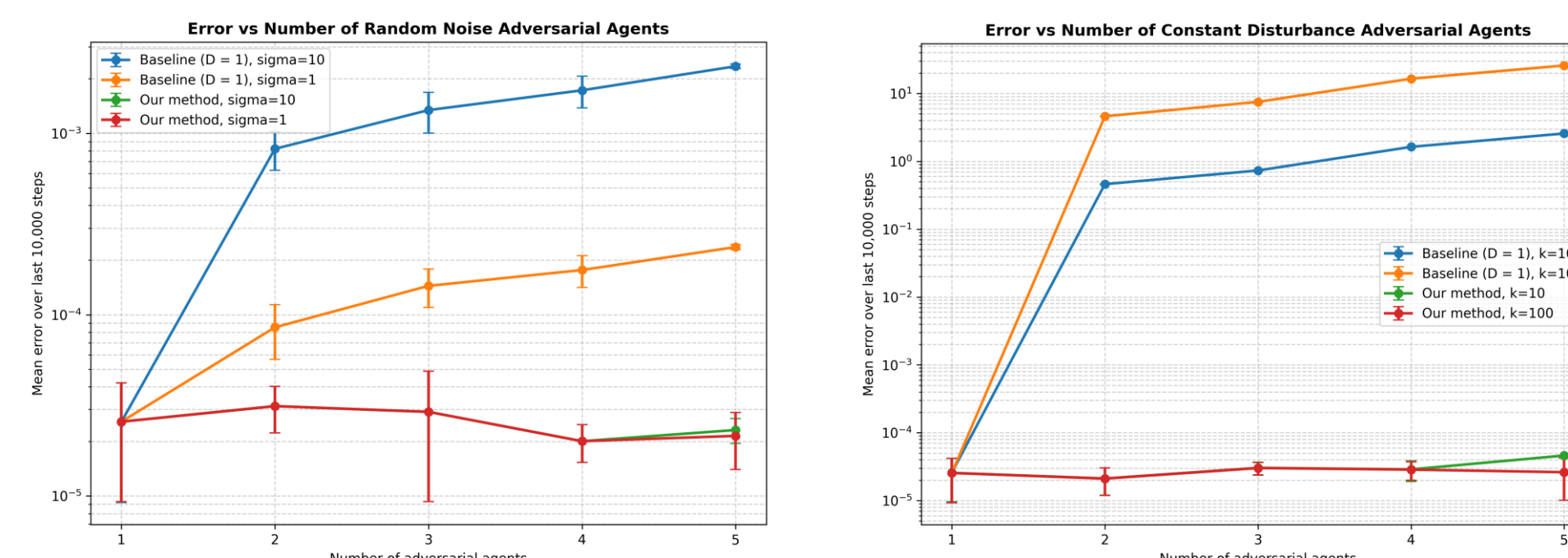


Figure 6. **Left:** Comparison using random noise adversarial agents, with different values for sigma. **Right:** Comparison using constant disturbance adversarial agents, using different values for k (all experiments done with 12 total agents)

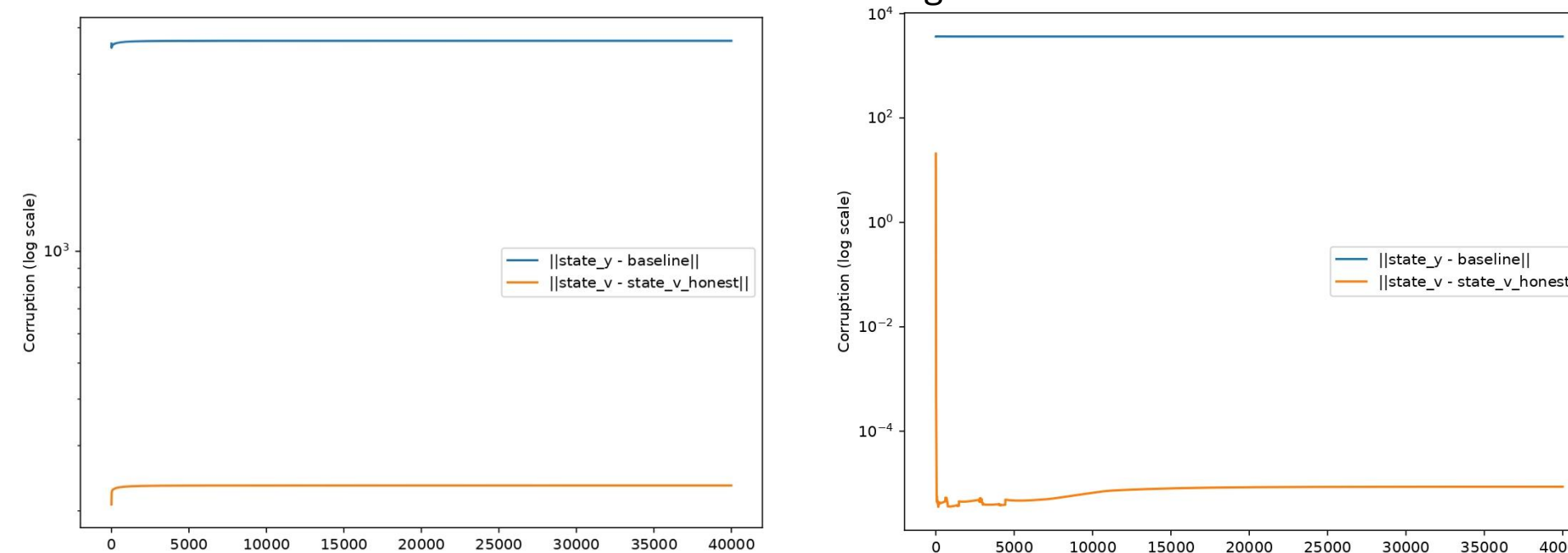


Figure 7. **Left:** The corruption removed by the baseline method. **Right:** The corruption removed by our method. As seen, our method removes much, much more of the adversarial contamination than the baseline method (plots are for constant disturbance, $k = 100$, 12 total agents)

Adaptive Gradient Play

The original algorithm uses a constant step size α for each gradient descent update. We implemented a modular adaptive step size framework supporting ADAM, AdaGrad and Nesterov Momentum. The Nesterov Momentum method achieves 10x faster convergence compared to the constant update method in our empirical results, shown in Fig. 8

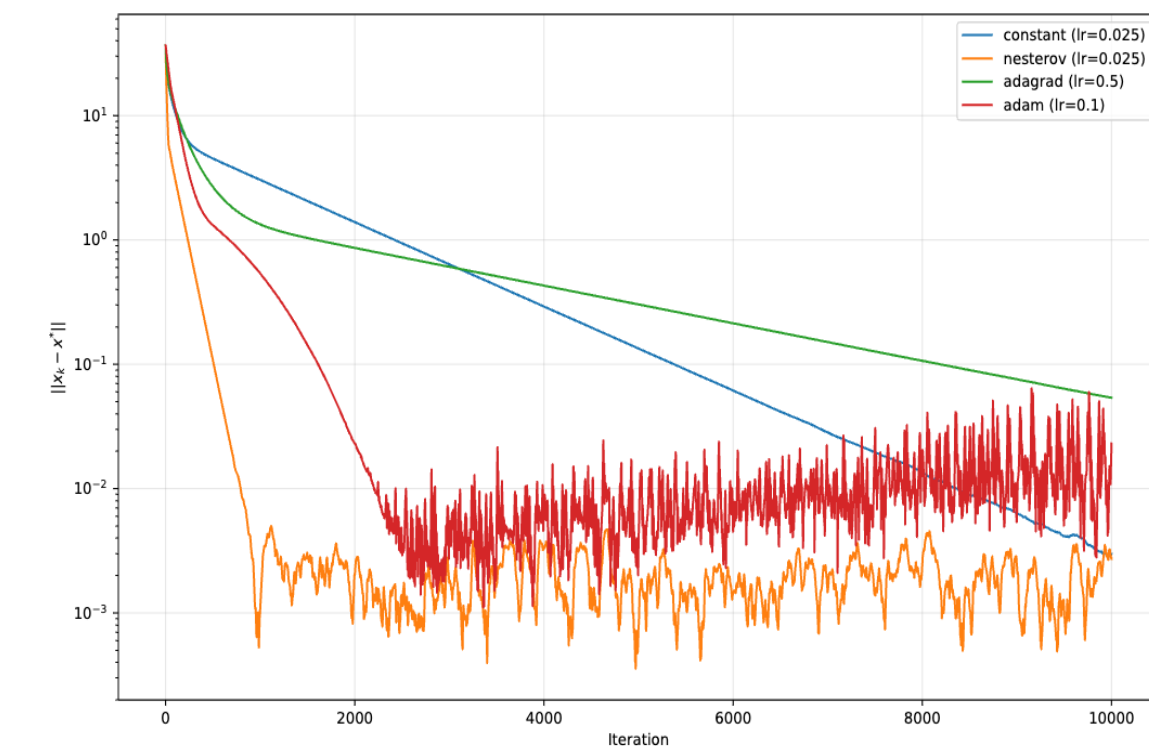


Figure 8. Number of iterations vs. Distance to NE for our implemented gradient update methods

Constrained Optimization Algorithm

To address the search space of constrained optimization, we introduce a projection step after gradient update to make sure the agent’s actions are within the feasible set.

The feasible set must be convex and in the Euclidean space for the system to be convergent without further assumptions. We simulate with ball and box feasible regions and compare them with unconstrained optimization shown below.

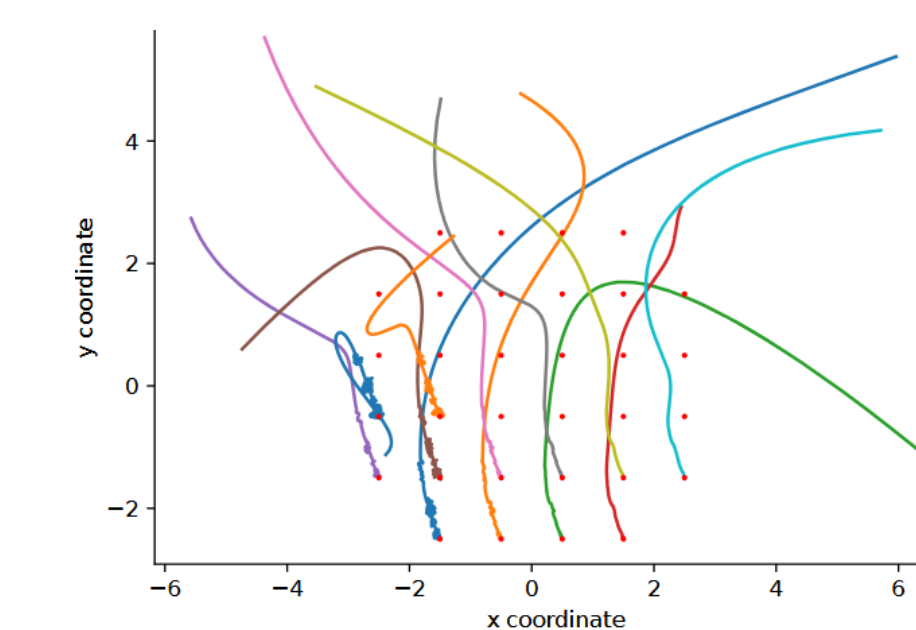


Figure 9. Unconstrained Optimization Example

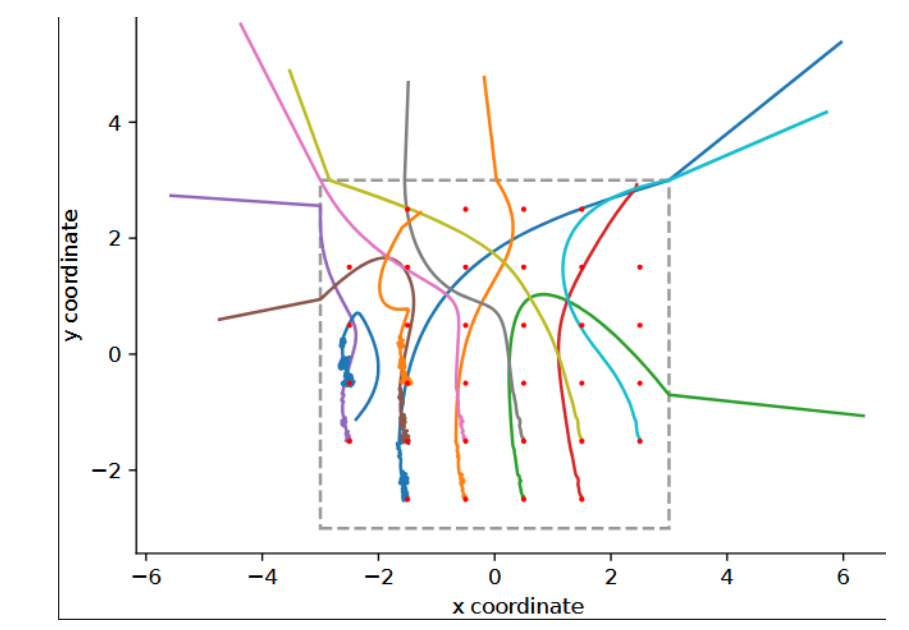


Figure 10. Constrained Optimization Example: Box

Time-Varying Networks & General Proof Structure

The analytical proof of the system under time varying influence follows the structure shown in Fig. 11

1. Address necessary assumptions about network topology:
 - a. Strongly Connected
 - b. Unrestricted Information Flow through only truthful agents
 - c. Majority neighbors are truthful

2. Show that after filtering the messages can be represented as in the convex combination of all messages.

3. Show that with unrestricted information flow through the graphs each agent’s information will reach other agents

5. Using all properties from previous lemmas, formulate a Lyapunov function and show the properties of lower energy based on algorithm’s traversal, giving Lyapunov stability and hence 6. Convergence!

Our empirical simulations use grid, ring graphs shown in Fig. 12

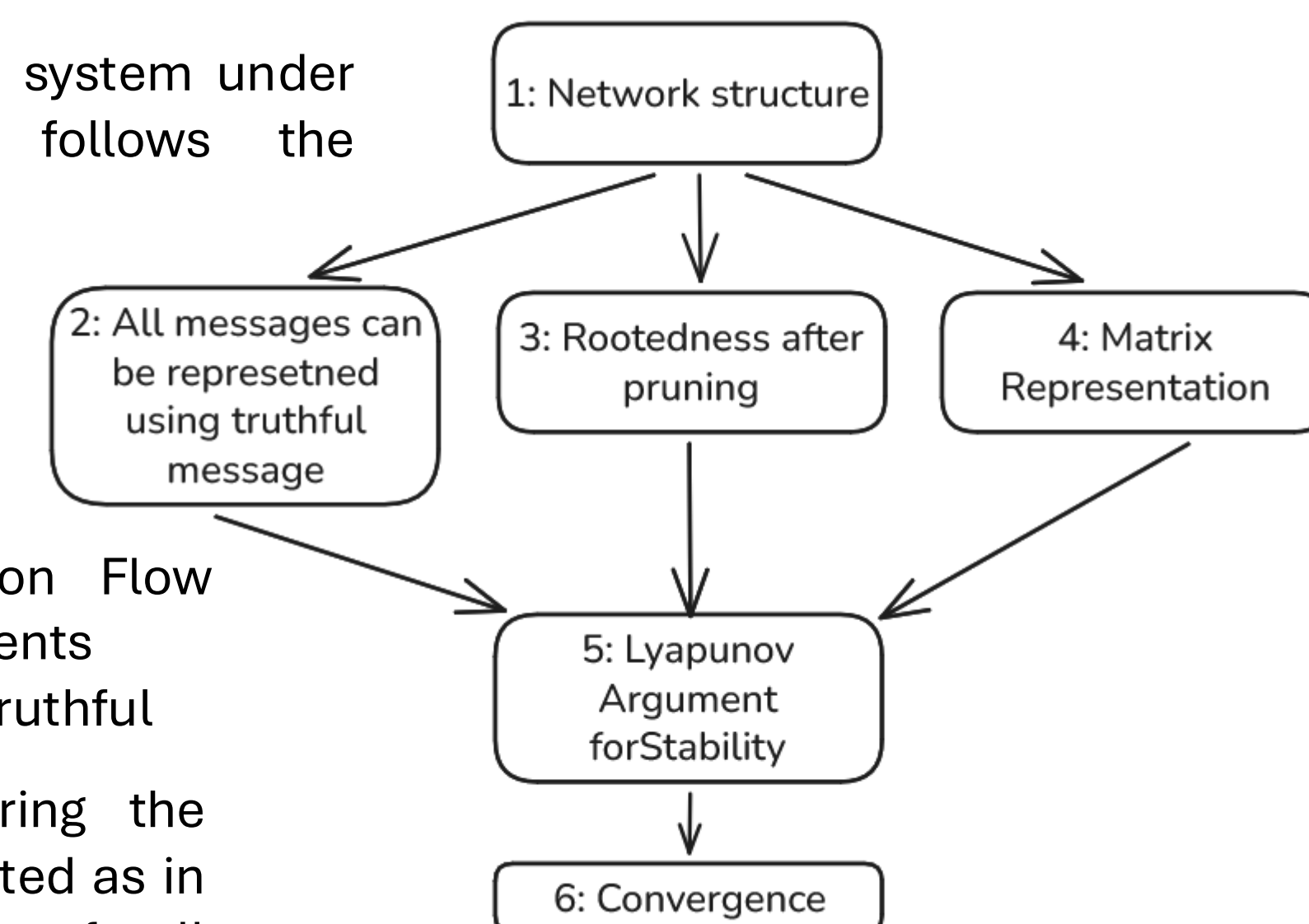


Figure 11. Proof Logical Flow of Analysis Setup

4. Show that the algorithm can be represented in matrix form and have convergent properties

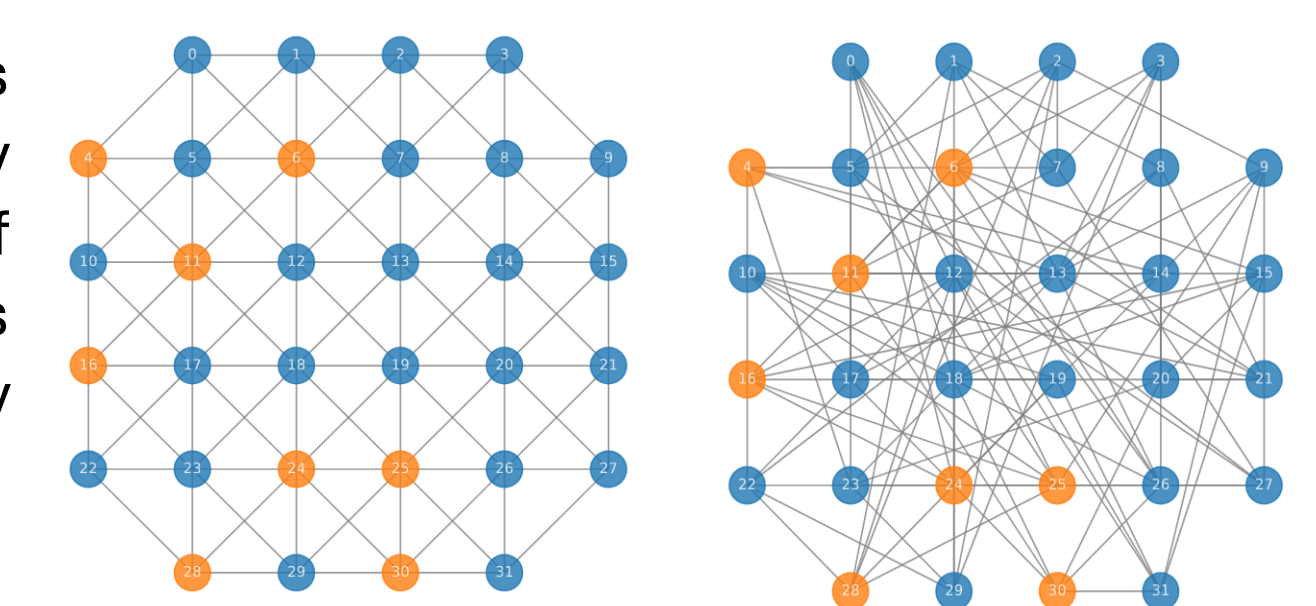


Figure 12. Time-varying communication graphs at different timesteps. **Left** Grid graph **Right** Ring Graph